



— TEST-TIME COMPUTE · 检索 · 冻结模型

搜索， 是一种 test-time compute

与其把模型换大，不如在推理这一刻多算几遍。
省下来的钱，能买到相关性，也能买到全新的能力。

肖涵

Elastic 副总裁

@hxiao · hanxiao.io

— 开场

2026 年了，搜索还有什么可做的？

模型这边

榜单上挤满了人，第一名和第十名差几个千分点。
换谁家的 embedding，效果都差不多。

链路这边

召回、精排、混合检索、查询改写——
该有的组件，十年前就都有了。

于是的结论

搜索是个成熟行业，剩下的都是工程活。
热闹在别处，不在这儿。

这几个月我一直在想这件事。
后来发现，我们可能一直站在一个不太对的角度看它。

搜索，本来就是一种 test-time compute。

看清这一点，很多原本觉得做到头了的地方，会重新变得有事可做。

— 先把概念说清楚

什么叫 test-time compute。

一句话：**不去训一个更大的模型，而是让手上这个模型，在回答每个问题时多干一点活。**训练阶段的钱已经花完了，这笔新的开销花在推理这一刻。

Best-of-N

一口气采样很多个答案，挑最好的那个交出去。

Self-consistency

让它把思路想好几遍，哪个答案出现得最多就用哪个。

Verifier search

拿一个判分器在候选里搜，愿意多花算力，成绩就往上走。

模型没变，权重一个没动。变的只是你愿意为这一次回答，多花多少算力。

— 这笔账有多划算

让机器人在一手扑克上多想 20 秒，effect 相当于把模型放大 10 万倍、再多训 10 万倍的时间。

Noam Brown, OpenAI, 2024

20 秒，换 10 万倍。

这个兑换比例一旦成立，问题就从「模型够不够大」，变成了「你舍不舍得在推理时多花这一点」。

01 | 为什么说搜索就是它

多跑几遍，跑得更深一点，然后看看能换回来什么

把搜索拆开看

你搭的每一条检索链路，都是在推理时多算几遍。

向量召回一遍，精排再来一遍，查询改写之后又是一遍，多路结果还要融合。这些环节没有一个是 在训练阶段完成的，**全都发生在用户按下回车之后**。这不就是 test-time compute 吗，只是我们一直管它叫「搭链路」。

A

Multi-pass：多跑几遍

同一个模型，换个角度再编码一次。把文档切成句子分别算，把图片切成局部分别看。**每一遍都会带回上一遍漏掉的东西。**

B

Deeper pass：跑得更深

同一次前向，不只取最后那个池化向量，把中间的 patch、token 全都读出来用。**算力几乎没多花，信息多拿了一大截。**

C

Compose：串起来

让 agent 自己决定先 grep 还是先 embed，要不要重排，要不要再搜一轮。**链路本身成了推理期的产物。**

最小的一个例子

从一次 cosine，到 late interaction，中间隔的全是算力。

单向量 COSINE



$$\cos(q, d)$$

一篇文档压成一个向量

起点

句子级 MAXSIM

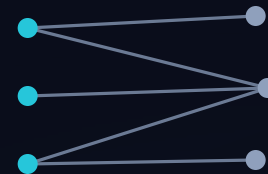


$$\max_s \cos(q, s)$$

还是那个模型，把文档切成句子各算一遍

这一步就是 TEST-TIME COMPUTE

COLBERT 多向量



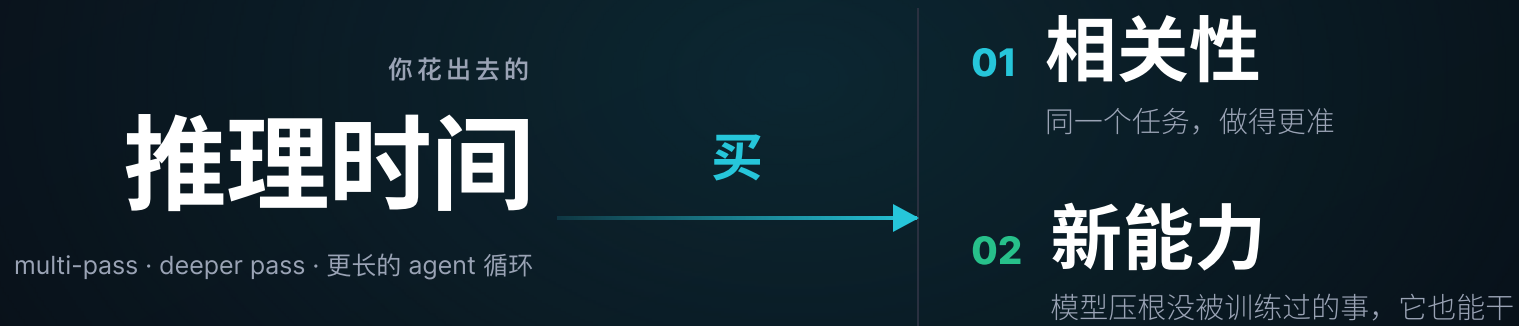
$$\sum_i \max_j \cos(q_i, d_j)$$

每个 query token 对全部 doc token 取 max

得换一个多向量模型

中间这一列最有意思：模型一个字节都没改，只是在推理时多算了几遍，就走到了原本得换模型才够得着的位置。

—— 这一页请记住



搜索的天花板，从来不在模型有多大。
在于你舍得为一次查询，花掉多少推理算力。

— 接下来

四个项目，都是这句话的注脚。

01

Autoresearch

让 agent 通宵在一个冻结的编码器上
搜程序，144 代。

它自己把经典 IR 重新发明了一遍。

买到的是 · 相关性

02

Image tagging

一个只会做检索的模型，被推理期的
算力改造成了开放词表的图像标注
器。

买到的是 · 新能力

03

Dataroom

用便宜的本地小模型把开放网络啃成
一个带引用的 zip，留着贵的 token 干
正事。

买到的是 · 召回

04

Searchbox + 知识图谱

把 agent 关进没有网的盒子里，只能
自己现搭链路。再用知识图谱给它出
多跳难题。

买到的是 · 精度

从模型内部一直做到 agent 链路，代码全部开源。

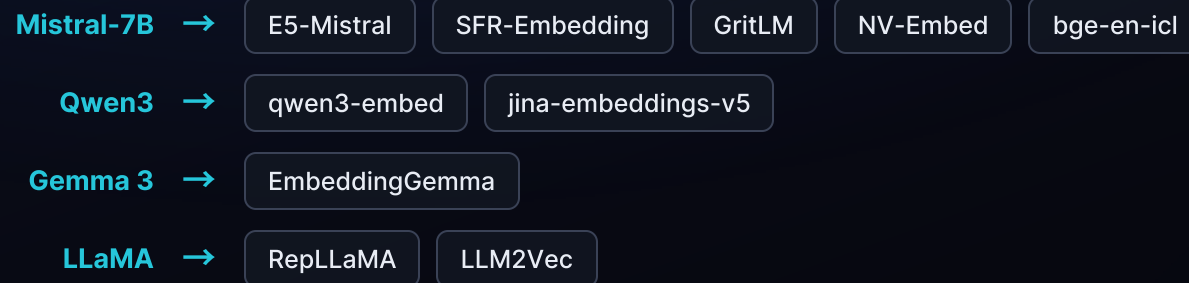
02 | Autoresearch: 让 agent 自己去找

冻结一个 239M 的编码器，通宵搜 144 代程序，看看推理期的算力能榨出多少相关性

一个容易被忽略的事实

今天的小向量模型，都是大模型的孩子。

大家默认 test-time compute 是大模型的专利，小模型没这个命。可你看看今天这些能直接上线的向量模型是从哪来的：



从大语言模型蒸出一个几百 M 的编码器，现在是主流做法，不是个别现象。

既然 test-time compute 住在 LLM 的表示空间里，**这些孩子多少该继承一点**。那就试试看。

— 规则先立好

只准动推理，不准动别的。

不重训

不请外援模型

不学任何新参数

手上只有一个冻结编码器的 API

规矩定这么死，是因为一松口，结论就没意义了。最近检索圈这些 test-time 方法，**基本上每个都得破一条戒**：

| **HYDE / QUERY2DOC**

查询路径里请了一个 LLM 来写假文档。

| **GQR**

得再养一个检索器在旁边做监督。

| **METAEMBED**

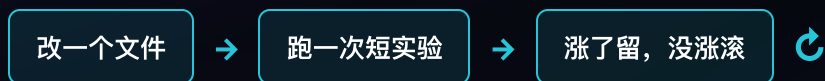
上线的时候还得训一批新参数。

这三条全禁掉。剩下的空间里，还能变出什么花样？

做法

这活我没自己干，我让 agent 通宵去爬这座山。

流程简单到有点土：**改一个文件**，**跑一次限定预算的短实验**，**指标涨了就留着，没涨就回滚**，然后一遍一遍来。说白了就是爬山法，只不过负责「变异」的那一步换成了一个 LLM。



144 代跑下来，那些 Python 程序**没有一行是我写的**。

这本身也是一次 test-time compute，只不过算力花在了「找方法」这一层，而不是「搜文档」那一层。

循环里的四个角色

一个负责出主意，一个负责打分，还有一个负责记账。



记忆会回流给出主意的那个：它读得到前面每一代干了什么、为什么没成

239M

发现阶段的编码器 jina-embeddings-v5-nano，小、快，循环才转得动

14

MMTEB Tier-1 任务，法律、金融、长文档、通用都有

144

程序总数，每一代的成本、父代、教训全部留档

— 先说清楚怎么算钱

成本就一个数：比基线多跑了几遍编码器。

$$c = 1 + \frac{\text{多出来的前向次数}}{\text{基线的前向次数}}$$

$c = 1$ · 一次都没多跑

比如 **SoftCentroid**：把已经算出来的 top-k 文档向量求个平均，掺回 query 里再打一次分。

$$q' = q + \text{mean}(d_emb[\text{top-k}])$$

全程只是在已有向量上做代数，编码器一次都没再启动。

$c > 1$ · 模型真的又跑了

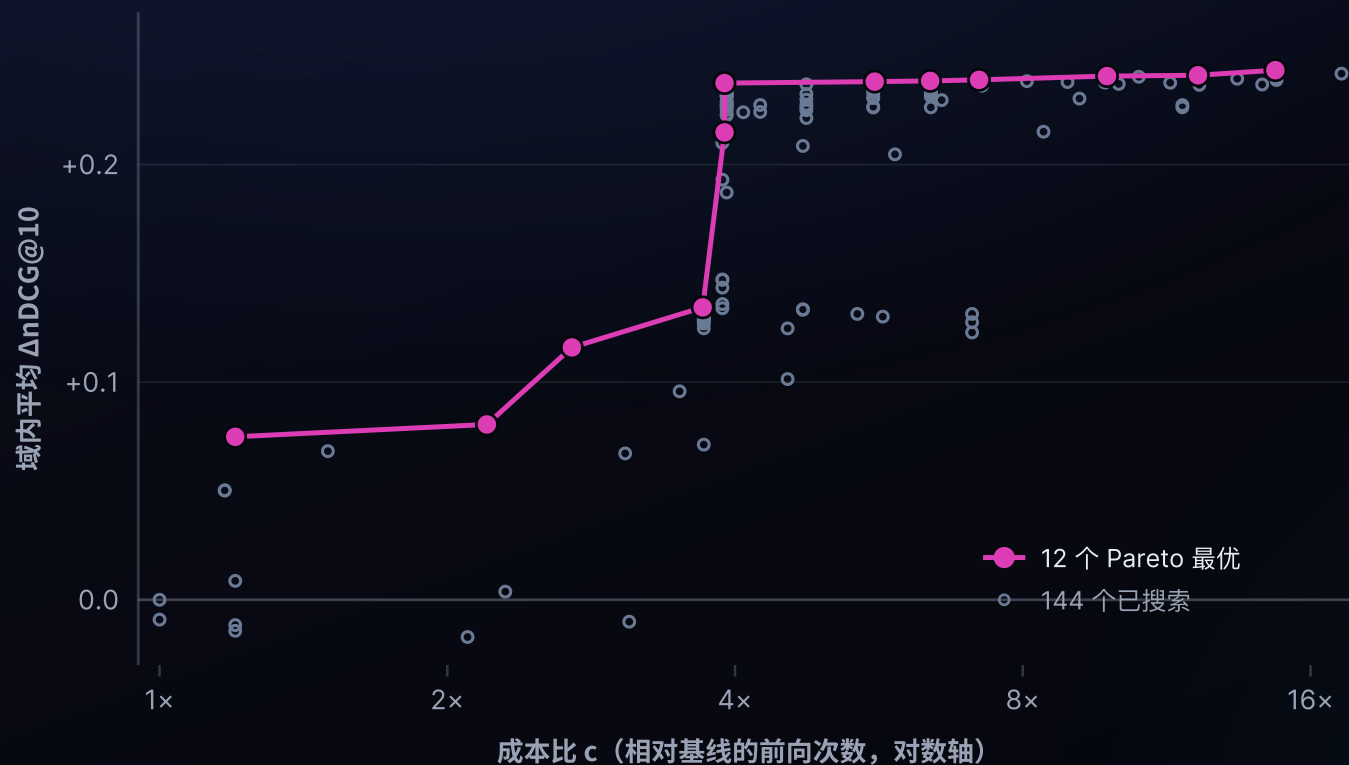
把文档切成句子重新编码，把 query 改写后重新编码，多轮反馈再来一遍——**每一次都得让模型真的再算一遍。**

这轮里最贵的程序 $c = 14.7$ ，相当于十五倍的推理开销。

这个区分很关键。后面你会看到，**这两类算力买到的东西不一样。**

第一个结果

放开了让它花算力，曲线爬得很漂亮。



144

144 代里搜出来的程序总数

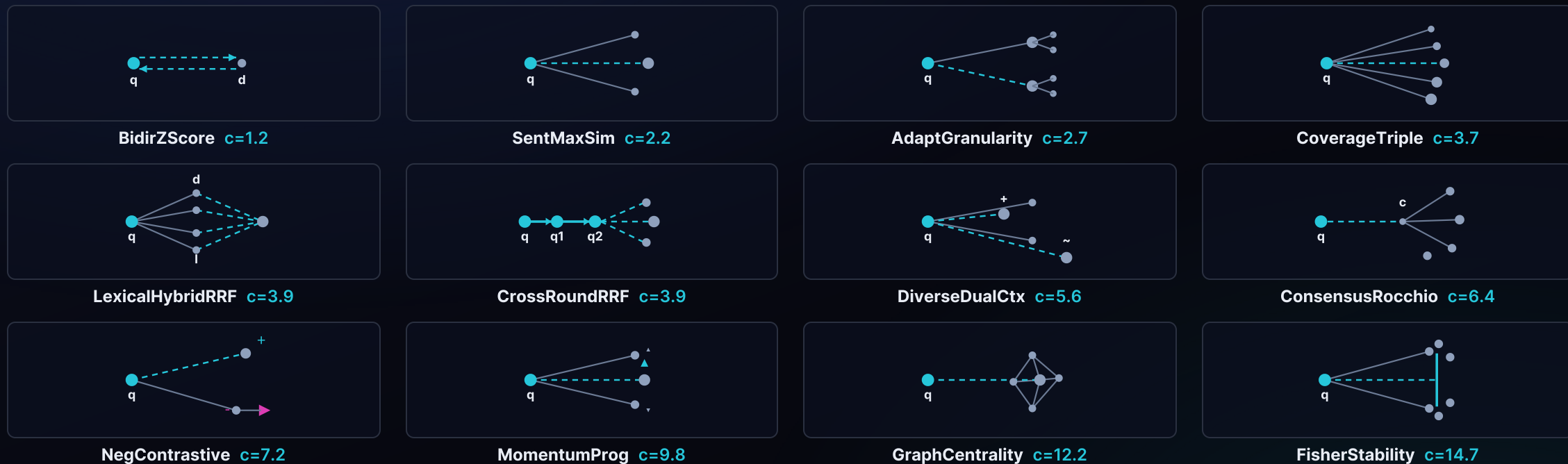
12

Pareto 最优的有 12 个，成本从 1.2 倍一直排到 14.7 倍

平均 $\Delta nDCG@10$ 从 +0.07 一路爬到 +0.24。这就是一条标准的 **test-time compute scaling 曲线**，只不过发生在一个 239M 的冻结小模型上。

它写出来的东西

这十二个程序，没有一个是人设计的。

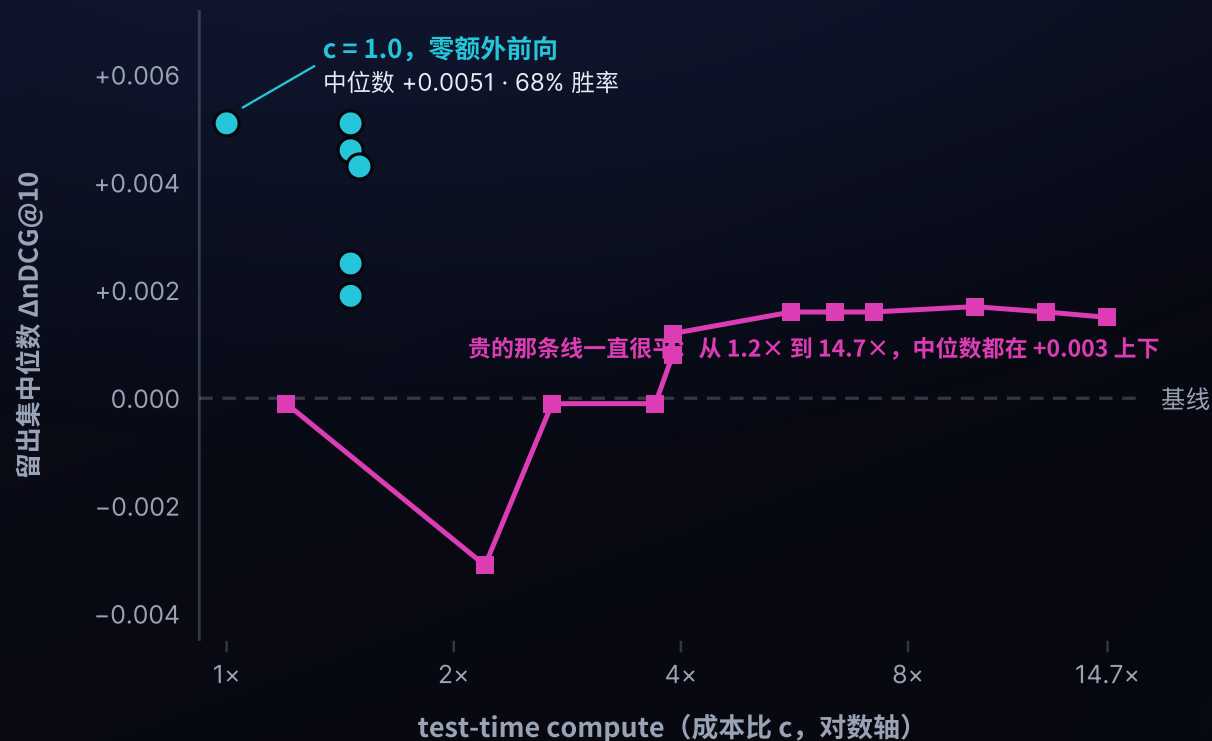


每一个都是对同一批冻结向量的免训练重组：切句子、做 z-score、融多路、拉质心、建图。成本从左往右递增。

更值钱的发现

最便宜的那批，反而能搬到别的模型上去。

按域内成绩收录的 (成本 1.2x → 14.7x) 要求能迁移才收的



换一套收录标准：只有在验证集上涨、并且**不把任何一个任务拖下水**，才准进门。

搜出来六个程序，成本全在 **c = 1.0 到 1.5** 之间。最好的那个 **Transpose-Consensus** 胜率 83.3%，**最差的任务是 0.0000**——一个都没弄坏。

真正能带走的东西，**往往不贵**。

六个能带走的程序

便宜、稳、而且换个模型照样管用。

程序	c	中位数	胜率	均值	最差的那个任务
ZeroCost-Consensus	1.00	+0.0051	68.4%	+0.0148	-0.1070
Deca-Axis	1.46	+0.0051	68.4%	+0.0133	-0.0365
Dodeca-v077	1.46	+0.0046	66.7%	+0.0126	-0.0420
Transpose-Consensus	1.50	+0.0043	83.3%	+0.0219	0.0000
Cross-Axis	1.46	+0.0025	64.9%	+0.0097	-0.0240
Penta-Axis	1.46	+0.0019	59.6%	+0.0107	-0.0485

这些都是纯粹的向量代数：不训练、不加参数、不请外援，换个编码器复制粘贴就能用。

搬去哪儿去了

最有意思的是，它在没见过的模型上涨得最多。

均值 中位数



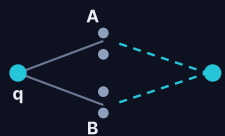
程序是在 **j-v5-nano** 上搜出来的。四个编码器上平均都是正的，而在 **gemma** 和 **qwen** 这两个完全没参与过发现的家族上，涨得最多。

这说明它抓住的不是某个模型的怪癖，而是**向量空间本身共有的几何结构**。所以才搬得动。

换句话说，这批程序对**你手上那个模型**大概率也有效。

它到底找到了什么

兜了一圈，它自己把经典 IR 重新发明了一遍。



Reciprocal Rank Fusion

REDISCOVERED, NOT SEEDED

fuse two rankings into one consensus



Fisher Linear Discriminant

REDISCOVERED, NOT SEEDED

the axis that best separates relevant from not



Rocchio Pseudo-Relevance Feedback

OPERATIONALIZED FROM A SEEDED IDEA

pull the query toward the relevant centroid



Sentence-level MaxSim

OPERATIONALIZED FROM A SEEDED IDEA

score the best sentence, not the mean

这些名字我一个都没告诉它。它是从几何里自己摸出来的——因为这些结构本来就长在向量空间里。

—— 这一段的收获

一个 239M 的冻结模型，靠推理还能再榨出这么多。

01 小模型也吃 test-time compute 这一套

144 代、12 个 Pareto 最优解， $\Delta nDCG@10$ 从 +0.07 爬到 +0.24。这条曲线过去只在大模型身上见过。

02 最划算的那一档，几乎不花额外算力

六个能迁移的程序全在 $c = 1.0$ 到 1.5，纯向量代数，换模型即插即用，在陌生编码器上收益还更大。

03 而这些方法，是 agent 自己通宵找出来的

RRF、Fisher 判别、Rocchio 反馈、句子级 MaxSim——经典 IR 的老朋友，被它在向量空间里重新推了一遍。

这一轮买到的是相关性。

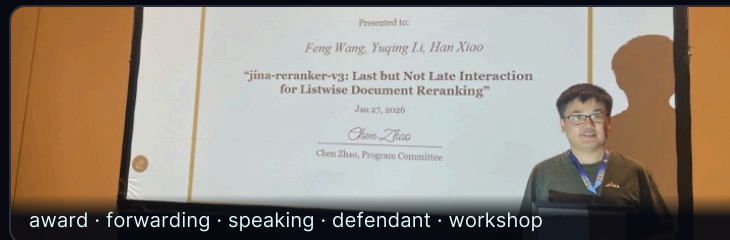
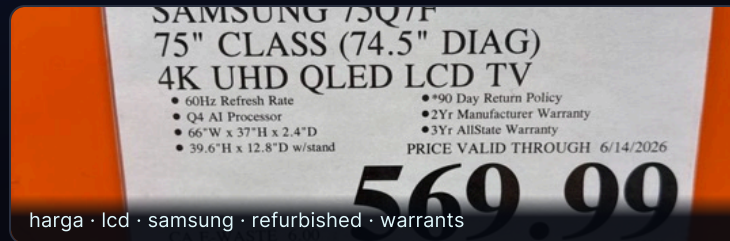
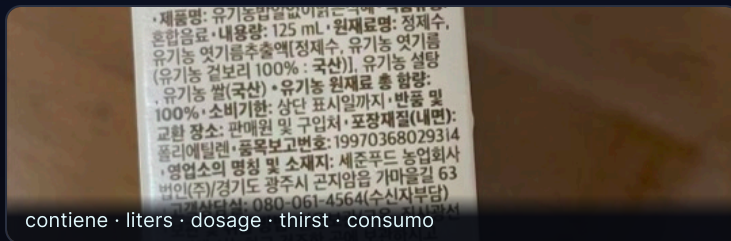
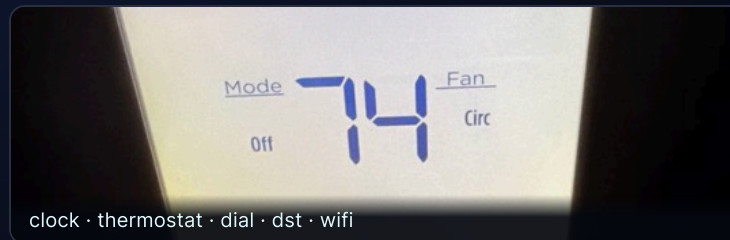
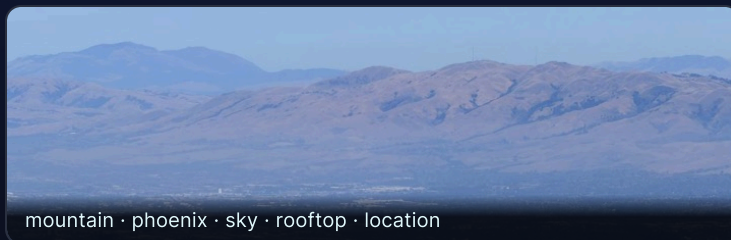
下一个项目我想试试：这笔钱能不能买到模型压根不会的东西。

03 | Image tagging: 买一个新能力

一个只会做检索的模型，从来没学过打标签。这次不求它更准，求它会一件新事

— 先看结果

这些标签，是一个检索模型吐出来的。



没训练、没请第二个模型、没查词典。全是推理期算出来的。顺带一提，它还是多语言的。

规则照旧

同样只准动推理，但这次要的是一件新差事。

- 手上有 一个 **jina-embeddings-v5-omni-nano**，大约 1B，图和文共用一个 768 维空间。
- 要它干 把图里**每一个**东西都说出来：多标签，而且词表是开放的。
- 前提是 权重全冻结。它是个检索编码器，**没有标注头，没有分类器，训练时压根没见过这个任务。**
- 只准用 它自己吐出来的那些数，在上面做推理期计算。别的一概不行。

不训练

不请第二个模型

不用 WordNet 和词典

不用正则和词性标注

这次的赌注比上一轮大：上一轮是**把会的事做得更好**，这一轮是**让它做一件根本不会的事**。

算力能怎么花

对一个向量模型来说，无非三种花法。

A

同一次前向，读得更深

别只拿那个池化完的向量，把 patch 级的输出全读出来，挨个跟 12.8 万个词打分。算力几乎没多花。

对应上一轮的 $c = 1.0$

B

换个视角，再跑一遍

把图切成若干 crop 重新编码。小东西在自己那张 crop 里能占满整个画面，就不会被忽略了。

对应上一轮的 $c > 1$

C

在已有结果上再做文章

白化、最优传输、图传播、EM——在**同一批像素**上做更花哨的数学。

这条我们也全试了

剧透一下：A 和 B 都真给了东西，而且给得比我预想的多。

— 四步走

把一个检索器，改装成标注器。

词表当标签空间

→

每个 patch 打分

→

减掉背景先验

→

去重取头部

01 tokenizer 的词表，直接拿来当标签空间

12.8 万个 token 全部编码成文本向量。不用人工整理标签集——模型认识多少词，就有多少个候选标签。开放词表是白送的。

02 每个 patch 单独打分，再跟全局向量融合

$0.7 \times \text{patch-max} + 0.3 \times \text{global}$ 。小物体不再被整张图的池化冲淡，mAP 从 0.264 直接到 0.635。

03 每个词减掉它自己的先验

高频词天生跟所有图都沾点边。把 μ_l 减掉之后，剩下的右尾才是真证据。

04 词首过滤，再在向量空间里做 NMS

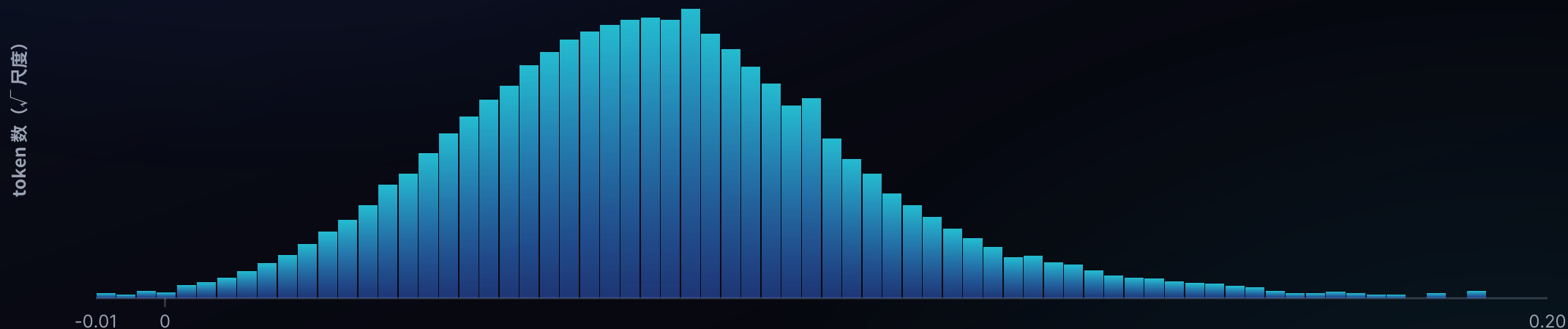
12.8 万压到 2.5 万候选；然后把 kitty / cat / kitten / chatte 这种同义词簇合并成一个概念。

— 这是真实的分布

12.8 万个词的得分，是怎么一步步被整理出来的。

1· 原始融合得分

全部 128,260 个 token · 0.7 patch-max + 0.3 global



一条光滑的钟形：每个词都跟每张图「有点像」

COCO-150

读得更深，加上多看几眼，就是这么两级台阶。



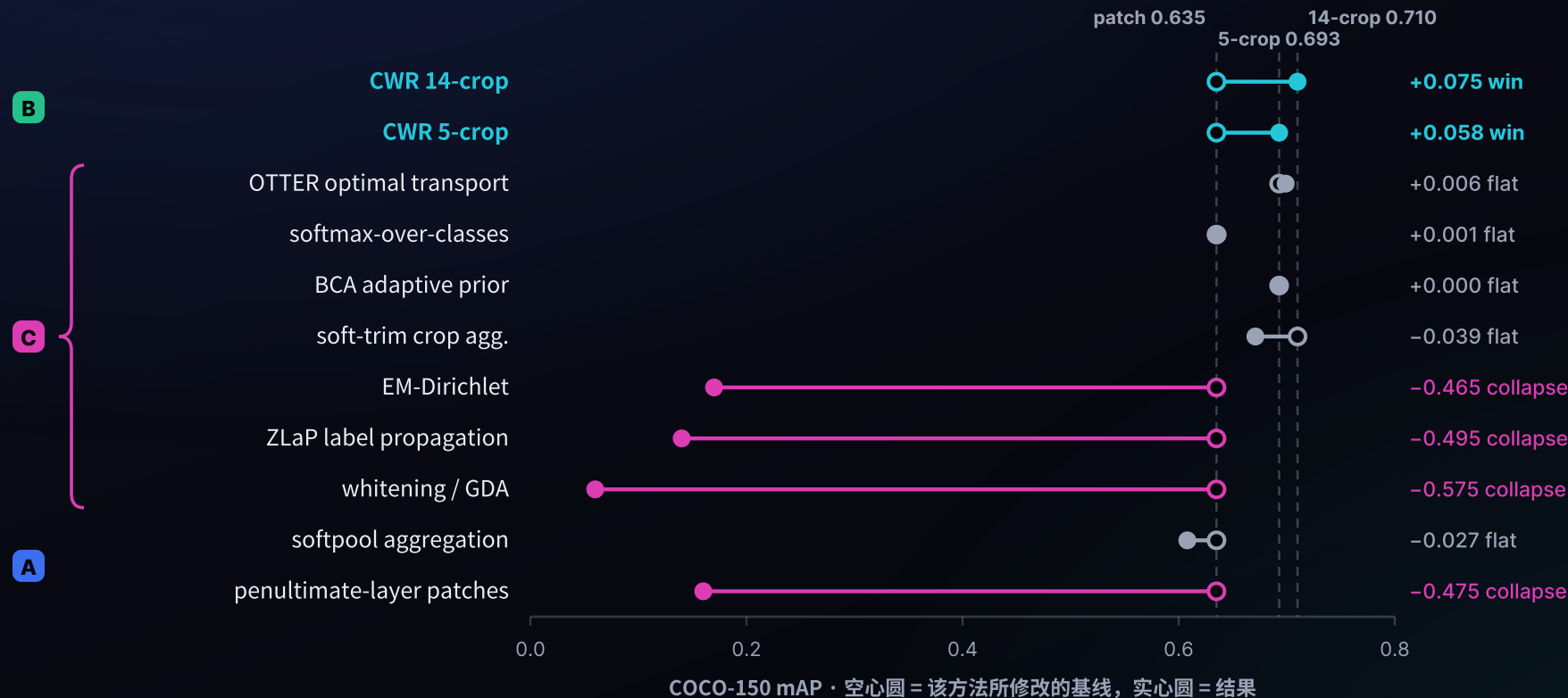
做法	P@1	P@3	R@5	MAP
global pooled	0.433	0.289	0.449	0.264
patch max + global	0.753	0.427	0.631	0.635
softpool	0.773	0.449	0.649	0.608
+ CWR multi-crop	0.813	0.476	0.680	0.710

只用池化向量是 0.264；改读 patch 之后 0.635；再多切几个 crop 重新编码，0.710。

A 类一步就 +0.371，B 类再补 +0.075。这中间模型一个权重都没动过。

我们试过的所有招

2024 到 2026 的免训练方法，挨个跑了一遍。



能站住的是 B 类：真的去看新的像素。A 类把这次前向读透。

至于 C 类那些更花哨的数学——它们的前提是图和文在两个空间里，可这儿本来就是一个空间。

算力和精度的关系

规律很干净：能带回新信息的算力，才换得到东西。



75 毫秒读透一次前向，1 秒钟看十五个视角。这条前沿线上的每一步，都是往系统里灌进了新东西。

— 已经上线了

这套东西现在跑在 Omni 里，一个本地的 macOS 搜索 app。

~4%

打标签的额外开销，搭在原本就要跑的 embedding 前向上顺手完成

0.847

精细模式（5-crop）P@1，Swift/bf16 端口，基线 0.773

32 帧

视频每 240 秒采样一次，跨空间和时间一起取 max

图片

建索引的时候顺手把标签也产出来，不用第二次前向。

视频

抽帧之后共用同一套 patch 打分逻辑，时间维直接并进 max。

扫描版 PDF

当图片处理。那些 OCR 认不出来的老档案，现在也能用词搜到了。

编码器、索引、存储全在那台存着文件的 Mac 上。从一篇实验记录到一个能用的功能，中间只隔着那 4% 的算力。

04 | 往上一层：让 agent 自己搭链路

前面两个项目都在模型内部。这一层，算力花在了 agent 的循环里

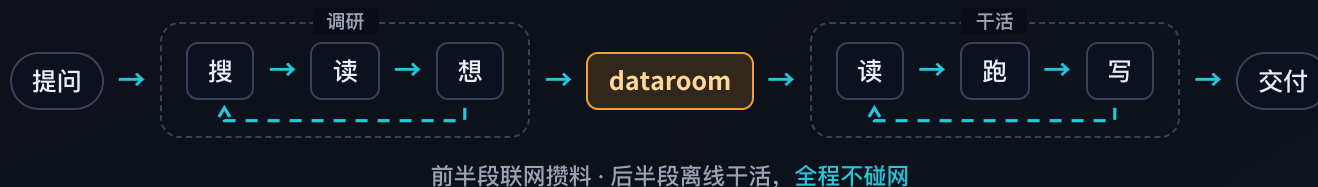
—— 这两年的变化

从 deep research 到长程任务，循环越来越长。

2025 · Deep research



2026 · 长程任务



调研和干活，该用的 token 不是一种。攒料用便宜的本地小模型，贵的额度留给真正需要它的那一段。

dataroom: 把开放网络啃成一个 zip。

GITHUB



给它一个 token 预算，它就开始**搜、读、写**，一轮一轮来，直到把这个题目下能找到的东西全部打包成一个**带引用的 zip**。

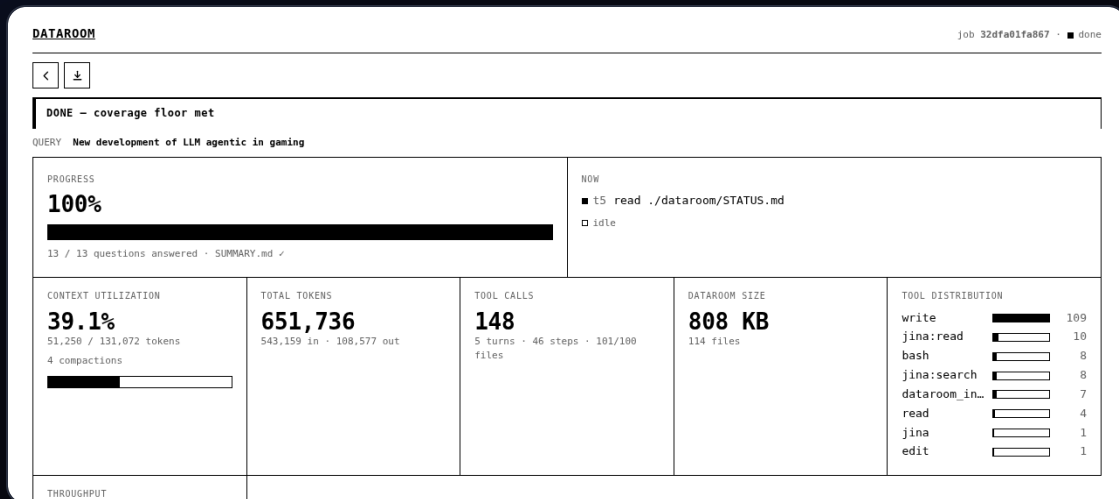
这个名字是我创业时留下的习惯——当年给投资人准备 data room，把该看的材料一次性理清楚放进去。现在换成给 agent 准备。

关键是 token 的账：探索阶段用本地的 Qwen 27B、35B-A3B 这类便宜模型跑，把昂贵的前沿额度省下来，留给后面真正要动脑子的那一步。

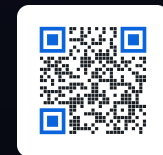
dataroom
攒出语料 (.zip)



searchbox
只从这个 zip 里找答案



任务实时页面：覆盖度进度、token 消耗、工具调用分布。停止条件看的是覆盖够不够，不是钱花完没有。



项目四 · 断网做题

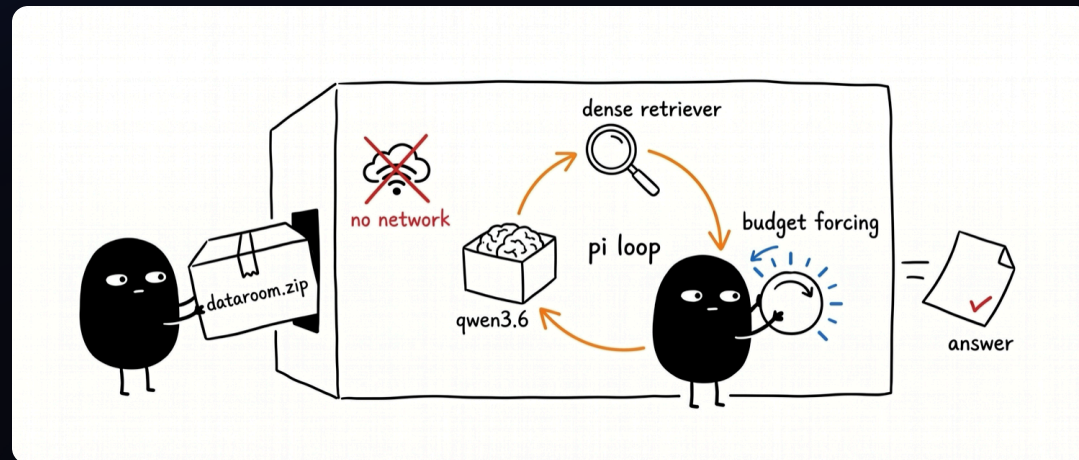
searchbox: 把 agent 关进没有网的盒子里。

丢给它一个 dataroom 的 zip，然后**切断网络**，再问一个问题。它想答上来，只能**用手边的本地工具现搭一条链路**——grep、embed、rerank、相似度、聚类、选多样性。

断网这件事是故意的。**不断网，你永远分不清它是真的在搜，还是网上碰巧有现成答案。**关上门，搜索能力才被逼出来。

正在研究的几个问题

- 它第一个伸手去拿的是哪个工具？
- 是不是 grep 就够了，稠密检索到底在哪些地方才真有用？
- 逼它多花算力，难题上会不会有起色？
- 它搭出来的链路，下次还能不能复用？



断网循环，画成漫画就是这样

knowledge-graph: 用知识图谱造多跳问题。

GITHUB

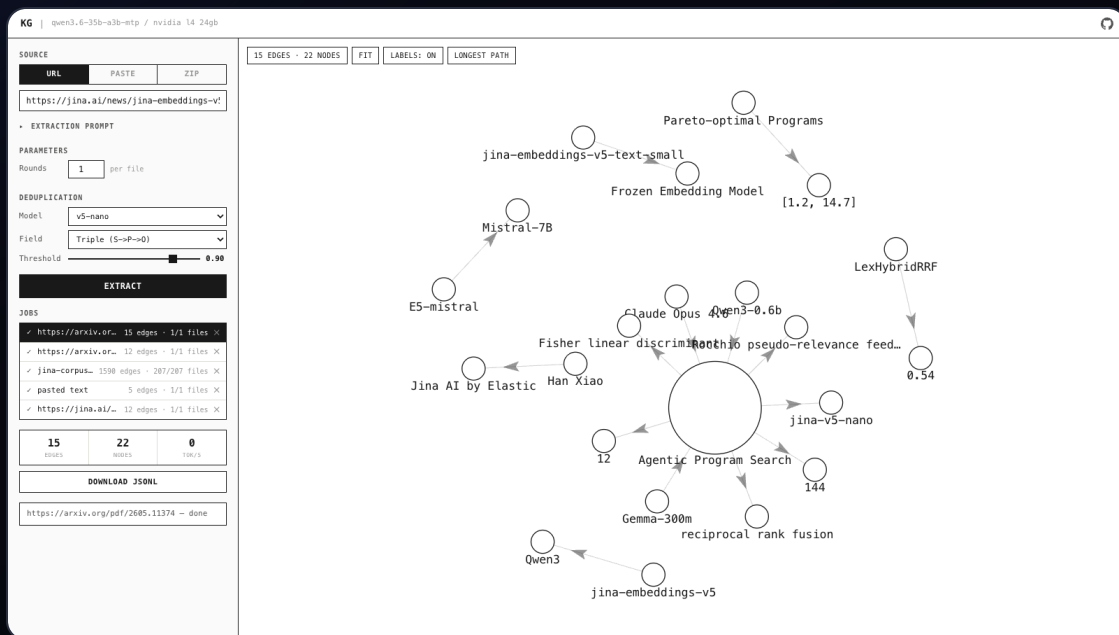


简单题对研究 agent 搜索毫无用处：**一次 grep 就能命中的问题，什么方法做出来都一样分。**要评测 searchbox，得先有能把它逼出汗的题。

怎么造

把语料先抽成知识图谱，每条事实是一条 (主语)-[谓语]->(宾语) 的边，然后去**走图里最长的那些路径**。这些长链条自然就成了**多跳问题**——没有任何单独一段文字能直接回答。

题目从同一份语料里长出来，所以是**私有的、不可能被背过的评测集**。agent 必须真的把事实一环一环连起来，也就必须真的去花那份推理算力。



实时图谱：每条事实一条边，最长路径视图会把多跳问题直接挑出来。

正在做的一个

dataroom harness：把检索单独拎出来测。

前面三个是完整的链路。想知道**检索这一环到底值多少**，得把它单独拎出来：固定一份 Jina 文档语料（210 篇 md 加结构化 json，切成 7777 个 chunk），外面套一个 agent，然后**只给它一个外部工具**。

其余全是 Pi 自带的 read、grep、bash。这样被测的就只有 search_corpus 这一件事，**功劳跟锅都没地方推**。

命中不是答案

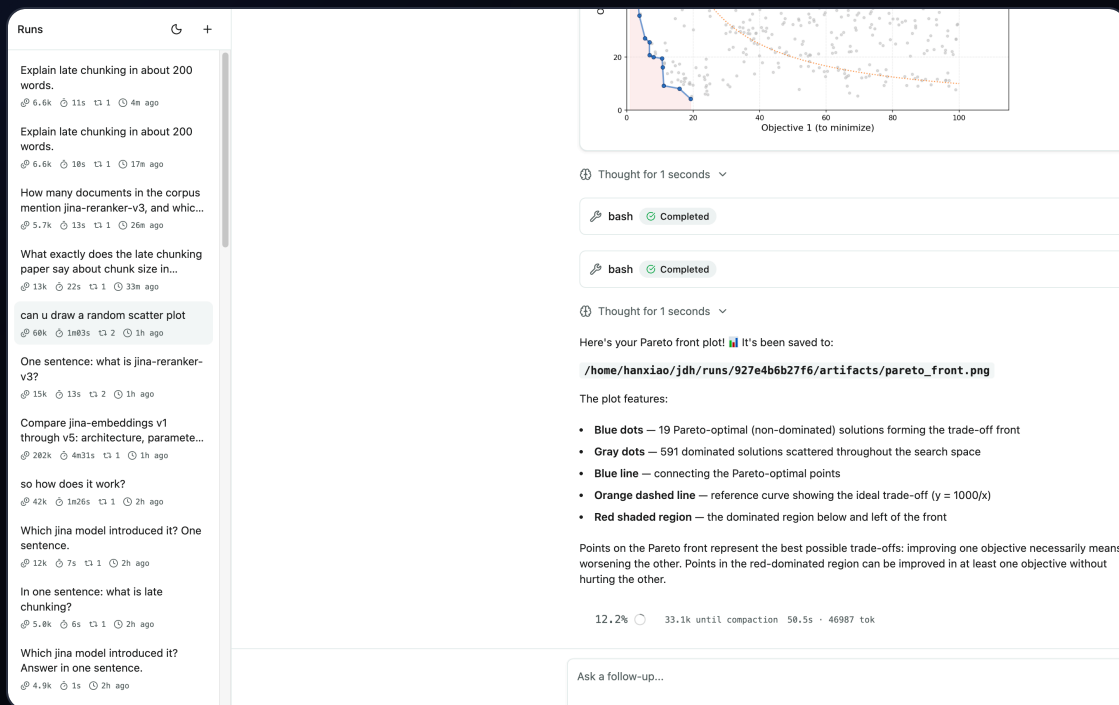
返回 {score, path, lines, text}，agent 得自己回原文把段落读完、再 grep 一遍看还有哪儿提到。

没有 SIDECAR

7777×768 的矩阵直接在扩展里算：加载 117ms，单次检索 4.3ms。少一个要启动、要占端口、会挂掉的服务。

联网是开关

默认关。开了才挂上 search_web 和 read_url，于是“翻书”和“上网”可以分开计分。



跑完一轮的样子：思考流、每一次工具调用及其参数、上下文占用、自动压缩、每段花了多少秒，全部流式开着。

— 这几个是一套

攒料、做题、出题，凑成一个能自己跑起来的闭环。

01

dataroom · 拉满召回

便宜的本地 token 把网络啃成一个带引用的 zip。这一步决定了天花板——材料里没有的东西，后面谁也变不出来。

02

searchbox · 拉满精度

断网，只给本地工具，逼 agent 在推理期自己组装链路。这一步决定了你能从材料里捞出多少。

03

知识图谱 · 说了算的裁判

从同一份语料长出多跳难题。有了它，前面两步的每一次改动才有得比。

04

harness · 单独拷问检索

固定语料，只留一个检索工具。把链路里那一环的贡献单拎出来看。

跟前面两个项目是同一件事，只是换了一层：那边组装的是向量代数，这边组装的是工具链。都没有把模型换大。

05 | 回到那句话

这几个项目跑完，我更相信这个判断了

五个项目，两个层次

一层在模型里面，一层在模型外面。

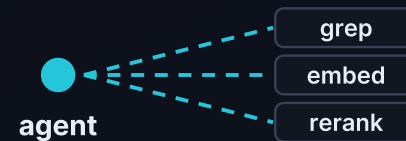


A 模型内：向量代数

推理时干的 多跑几遍，读得更深

项目 autoresearch · image tagging

买到的 相关性 + 一个全新的能力



B 模型外：工具链

推理时干的 自己决定先搜什么、再用什么

项目 dataroom · searchbox · 知识图谱 · harness

买到的 召回 + 精度

两层，一个赌注：多花推理算力，而不是换个更大的模型。

— 我现在怎么看这件事

三个我自己拿来判断的问题。

01 先问：这次多花的算力，带回了什么新东西？

被池化丢掉的 patch，被缩放丢掉的像素，被单向量抹平的句子结构，被一次 grep 漏掉的那几跳。能说出带回了什么，这笔钱就花得值。

02 再问：最便宜的那一档，是不是已经够了？

autoresearch 里最能迁移的程序在 $c = 1.0$ 到 1.5 ；image tagging 里 A 类一步就吃掉了 $+0.371$ 。先把这次前向读透，往往比再跑十遍划算。

03 最后问：这条链路，能不能让机器自己去找？

144 代程序没有一行是我写的。方法本身也可以是 test-time compute 的产物。

按这三条走，2026 年的搜索远远没到没事可做的地步。

搜索是一种 **test-time compute**。
别急着换更大的模型，
先在推理时**多搜**一点。

而且这条链路你都不必自己设计——
让 agent 通宵替你找，第二天早上来看结果就行。

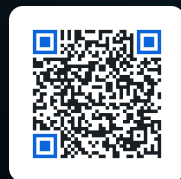


THANKS

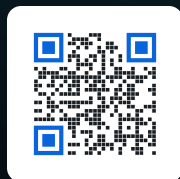
肖涵 · Elastic 副总裁

@hxiao · github.com/hanxiao · [arXiv:2605.11374](https://arxiv.org/abs/2605.11374)

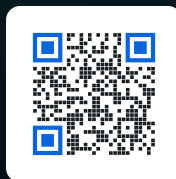
IMAGE TAGGING



DATAROOM



SEARCHBOX



KNOWLEDGE-GRAPH

